Ders 2 Genişletilmiş Notu: Dağıtık Dosya Sistemleri ve MapReduce Programlama Modeli

Ders 2 Genişletilmiş Notu: Dağıtık Dosya Sistemleri ve MapReduce Programlama Modeli

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu ve Kaynakları
- 2. Büyük Veri Neden Tek Makineye Sığmaz?
- 3. Küme Hesaplama Ortamı
- 4. Küme Hesaplamanın Sorunları
- 5. MapReduce'un Çözdüğü Üç Sorun
- 6. Dağıtık Dosya Sistemi
- 7. Dağıtık Dosya Sisteminin Bileşenleri
- 8. Kelime Sayma Problemi
- 9. MapReduce Adımları: Map, Group by Key, Reduce
- 10. Başka MapReduce Örnekleri
- 11. Arka Planda Olanlar
- 12. Ana Düğümün Koordinasyonu ve Arızalar
- 13. Map ve Reduce Görevi Sayısı
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Konusu ve Kaynakları

Hoca derse bu haftanın konusunu söyleyerek başladı: MapReduce ve MapReduce programlama modelini gerçekleştirmek için kullanılan yazılım yığını (software stack). Konu, ders kitabının (Mining of Massive Datasets, MMDS) 2. bölümüdür. Bölümün PDF'i, PowerPoint slaytları ve video dersleri ders web sitesinde bulunur. Anlatım çoğunlukla İngilizcedir. Soruların cevapları ve dersin ikinci yarısının bir kısmı Türkçedir.

Ders iki büyük parçadan oluştu. Önce dağıtık dosya sistemi (distributed file system, DFS) ve onu gerektiren küme ortamı anlatıldı. Ardından bu altyapının üzerinde çalışan MapReduce programlama modeli, bir kelime sayma örneği üzerinden adım adım işlendi. Ortada 10 dakikalık bir ara verildi.

2. Büyük Veri Neden Tek Makineye Sığmaz?

2.1 CPU, Bellek ve Disk

Hocaya göre büyük veriden söz ettiğimizde, tek bir makinenin belleğine sığmayan veriden söz ediyoruz. Günlük ofis işleri için kullandığımız iş istasyonları, dizüstü ve masaüstü bilgisayarlar, fiziksel belleğe sığan verilerle çalışır. Uygulamalar fiziksel belleği kullanır ve CPU üzerinde çalışır. Uygulama belleğe sığmayan bir veriye ihtiyaç duyarsa diske gider ve o veriyi belleğe getirir.

Buradaki temel nokta şudur: CPU'nun işlediği veri fiziksel bellekte olmak zorundadır. CPU ile disk arasında doğrudan bir bağlantı yoktur. Uygulamanın bir veriyi işlemesi için o verinin diskten alınıp belleğe konması gerekir.

> **Hocanın vurgusu:** CPU yalnız fiziksel bellekteki veriyle çalışır. Belleğe ve tek makineye sığmayan veri birden fazla makineye bölünmeli, sonra bu veriyi dağıtık bir hesaplama ortamında işlemenin yolları bulunmalıdır.

Hoca, diğer derslerde öğrenilmiş olabilecek klasik veri madenciliği ve makine öğrenmesi yöntemlerini hatırlattı. Bu derste ise veri madenciliği, makine öğrenmesi ve istatistik yöntemlerinin birden fazla makinede çalışan biçimleri incelenecek.

2.2 Google Örneği: 200 TB'ı Diskten Okumak

Hocanın sayısal örneği Google'dan geliyor.

- **Verilen:** 10 milyar web sayfası, her sayfa yaklaşık 20 KB. Disk okuma hızı 50 MB/s.
- **Toplam boyut:** $10 \times 10^9 \times 20$ KB $= 200 \times 10^{12}$ B $= 200$ TB.
- **Okuma süresi:** 200×10^{12} B $/\, 50 \times 10^6$ B/s $= 4 \times 10^6$ s (ara adım eklendi).
- **Gün olarak:** $4 \times 10^6 / 86400 \approx 46{,}3$ gün. Hoca "46 günden fazla" dedi.

Hoca, fiziksel bellekte hiçbir sınır olmasa bile, yalnız bu veriyi diskten belleğe taşımanın 46 günden fazla süreceğini söyledi. Yani bellek sınırı olmasa bile disk okuma hızı bir darboğazdır. Bu yüzden küme hesaplama ortamları kullanılır.

> **Ders sonrası not:** Hoca disk okuma hızını "50 megabit per second" dedi. 46 gün sonucu 50 megabayt/s (MB/s) ile tutar. 50 megabit/s ile hesaplansaydı süre $4 \times 10^6 \times 8/86400 \approx 370$ gün olurdu.

3. Küme Hesaplama Ortamı

Hoca slaytta bir küme hesaplama (cluster computing) ortamının resmini gösterdi. Resimdeki her dolap bir raf kabinidir (rack cabin). Her kabinde sıradan (commodity) sunucular bulunur. Bunlar genelde Linux çalıştıran makinelerdir.

- **Kabin içi bağlantı:** Kabindeki sunucular, kabinin içindeki bir anahtar (switch) ile birbirine bağlanır.
- **Kabinler arası bağlantı:** Kabinler de bir omurga anahtar (backbone switch) ile birbirine bağlanır.
- **Kabin büyüklüğü:** Her raf kabininde 16-64 sıradan Linux düğümü (node) bulunur.

Sunucular arasında böyle bir bağlantı kurulduğunda bir küme hesaplama ortamı elde edilir. Büyük ölçekli veri bu sunuculara dağıtılır. Erişilebilirliği (availability) sağlamak için veri dağıtılırken çoğaltma (replication) da yapılır. Bir sunucudaki veri parçası başka sunucularda da kopyalanır. Böylece sunucuya ya da bütün kabine bir şey olsa bile o veri parçası başka kabinlerdeki sunucularda bulunabilir.

4. Küme Hesaplamanın Sorunları

4.1 Düğüm Arızaları

Veriyi parçalara bölüp birçok sunucuya dağıttığımızda, bu sunucuların zamanla bozulma olasılığıyla uğraşmamız gerekir. Hoca bunun nedenini sunucuların niteliğiyle açıkladı: Kümede kullanılan sıradan sunucular özel tasarlanmış makineler değildir. Masaüstü ya da dizüstü bilgisayarda kullanılan anakart ve kartların aynısını kullanırlar. Bu yüzden düğüm arızası (node failure) olasılığı hesaba katılmalıdır.

Hocanın sayısal akıl yürütmesi:

- **Verilen:** Üreticiler bir sunucunun yaklaşık 3 yıl, yani yaklaşık 1000 gün dayanacağını söyler. Bu, her sunucuda yaklaşık 1000 günde bir arıza demektir.
- **1000 sunuculu küme:** $1000/1000 = 1$, yani günde yaklaşık 1 arıza.
- **1 milyon sunuculu küme:** $10^6/1000 = 1000$, yani günde yaklaşık 1000 arıza.

Günde 1000 arıza ciddiye alınması gereken bir durumdur. Bundan iki sorun doğar:

- **Veri kaybı:** Bir veri yalnız bir sunucuda duruyorsa ve başka hiçbir sunucuda kopyası yoksa, o sunucu bozulduğunda veriye bir daha erişilemez. Veri kalıcı (persistent) ve erişilebilir biçimde saklanmalıdır.
- **Uzun süren hesaplamalar:** Bir uygulama bir iki saat sürebilir. Hocanın örneği, bir görüntüyü işleyen, başka bir biçime dönüştüren ya da derin öğrenme hesabı yapan bir uygulamadır. Uygulamayı çalıştıran düğüm çalışma sırasında bozulursa, hesaplama başka bir düğümde yeniden başlatılmalı ve tekrarlanmalıdır.

Büyük veri üzerinde uygulama çalıştıran her dağıtık sistem bu iki konuyu çözmek zorundadır.

4.2 Ağ Darboğazı

Dağıtık dosya sistemleri bu iki soruna çözüm bulurken bazı darboğazlarla karşılaşır. Birincisi ağ darboğazıdır (network bottleneck). Sunucular anahtarlarla bağlıdır ve anahtarların belirli bir veri aktarım hızı vardır:

- Kabin içindeki anahtar, kabindeki herhangi iki düğüm arasında 1 Gbps hız sağlar.
- Omurga anahtar, kabinler arasında 10 Gbps'e kadar hız sağlar.

Hoca, 1 Gbps'lik anahtarla aynı kabindeki bir düğümden diğerine büyük bir veriyi taşımanın yaklaşık bir gün sürdüğünü söyledi. Bu darboğazın çözümü, hesaplamayı verinin bulunduğu yere taşımaktır.

> **Ek bilgi (derste anlatılmadı):** Kitabın slaytlarında bu örnek büyük olasılıkla "1 Gbps ile 10 TB veriyi taşımak yaklaşık 1 gün sürer" biçimindedir. $10 \times 10^{12} \times 8$ bit $/\, 10^{9}$ bit/s $= 80\,000$ s, yani yaklaşık 22 saattir. Bu, hocanın "yaklaşık bir gün" sözüyle uyumludur.

4.3 Dağıtık Programlamanın Karmaşıklığı

İkinci zorluk, dağıtık programlamanın anlaşılmasının zor olmasıdır. Dağıtık kod yazmak için alttaki hesaplama ve ağ altyapısını hesaba katmak gerekir. Bu yüzden hem istediğimiz programı yazmamıza yardım eden hem de alttaki ağ yapısının karmaşıklığını gizleyen programlama modelleri gerekir. Amaç, ağ yapısını ve soket programlamayı anlamakla uğraşmadan yalnız kendi kodumuza odaklanabilmektir.

Özetle iki şey gerekir: ağ darboğazını çözmek için hesaplamayı veriye taşımak ve dağıtık programlamayı basitleştirmek.

5. MapReduce'un Çözdüğü Üç Sorun

MapReduce, bu sorunlara çözüm bulmak için ortaya konmuş bir programlama modelidir. Hoca üç katkıyı sıraladı:

- **Kalıcılık ve erişilebilirlik:** Veri birden fazla düğümde yedekli (redundant) saklanır. Bu kalıcılık (persistency) ve erişilebilirlik sağlar.
- **Veri hareketini en aza indirmek:** Hesaplama verinin yakınına taşınır. Ağ darboğazı hatırlanırsa, veriyi düğümler arasında taşımak çok zaman alır. MapReduce hesaplamayı veriye yaklaştırır.
- **Karmaşıklığı gizlemek:** MapReduce basit bir programlama modelidir ve dağıtık hesaplamanın karmaşıklığını gizler.

  Hocanın vurgusu: Veriyi hesaplamaya değil, hesaplamayı veriye taşı. Kodun boyutu verinin boyutundan çok küçüktür.

6. Dağıtık Dosya Sistemi

6.1 DFS'nin Sağladıkları

MapReduce'un ayrıntılarına girmeden önce hoca dağıtık dosya sistemlerine baktı. Bir DFS üç şey sağlar: dosya ad alanı (file namespace), yedeklilik (redundancy) ve erişilebilirlik (availability).

- **Dosya ad alanı:** DFS'ye bir dosya konduğunda, sistem bu dosyaya tekil (unique) bir kimlik verir. Dosyaya erişmek istediğinizde bu kimliği DFS'ye verirsiniz. DFS dosyanın parçalarını bulur, bir araya getirir ve bütün dosyayı size döndürür.
- **Yedeklilik:** DFS dosyanın her parçasını birden fazla kopya olarak saklar. Kopyalar farklı sunucularda olduğu için hesaplama parçalara bölünebilir ve kod verinin bulunduğu yere taşınıp orada çalıştırılabilir.
- **Erişilebilirlik:** DFS'nin sunucularından biri bozulsa bile dosyaya hâlâ erişilebilir.

Örnek gerçekleştirmeler: Hadoop dağıtık dosya sistemi (Hadoop Distributed File System, HDFS) ve Google'ın dosya sistemi (Google File System, GFS).

6.2 DFS'de Hangi Veri Tutulur?

Hoca DFS'nin belirli veri türleri için kullanıldığını söyledi ve ilişkisel veritabanlarıyla karşılaştırdı. MSSQL, MySQL, PostgreSQL gibi ilişkisel veritabanları tek başına çalışan SQL veritabanlarıdır. DFS'de ise genelde şu tür veri tutulur:

- **Çok büyük dosyalar:** Toplam boyut yüzlerce gigabayttan terabaytlara, belki petabaytlara çıkıyorsa DFS iyi bir seçimdir. Hocanın örnekleri Dropbox, Google Drive ve Microsoft OneDrive'dır. Bir kişinin sakladığı veri küçüktür. Ama bütün topluluk verisini bu dağıtık depolara koyunca toplam boyut çok büyür.
- **Nadiren yerinde güncellenen veri:** Hocanın örneği log dosyasıdır. Log, olmuş bir olayın sonradan tutulan kaydıdır (after-the-fact data). Gerçek hayatta bir şey olduğunda bunu not eder, bir dosyaya kaydederiz. Geçmişte ne olduğunu öğrenmek istediğimizde bu log dosyalarını analiz ederiz. Olmuş bir şeyi geri gidip değiştirmek istemeyiz. Bu yüzden büyük veri genelde yerinde nadiren güncellenen veridir.
- **Sona ekleme:** DFS'ye yeni veri eklemek istediğimizde veriyi mevcut verinin sonuna ekleriz (append). Var olan veriyi geri gidip güncellemeyiz.
- **Ana işlem okumadır:** Büyük veri dünyasında yapılan temel işlem okumadır. Örneğin son 6 ayın log dosyalarına gidip sayma işlemi yaparız. Dosya miktarı çok büyük olduğu için bu sayma işlemi büyük ölçekli bir dağıtık dosya sisteminde yapılmalıdır.

Buna karşılık kayıt sistemi ya da sipariş sistemi gibi operasyonel veritabanlarında kullanıcı bilgisi değişir. Adresinizi, telefon numaranızı değiştirebilirsiniz. Operasyonel veritabanlarındaki her şeyin zamanla güncellenmesi gerekir. Büyük veri ise etkinlikleri kaydeder.

Bu bölümde hocaya iki soru geldi. Biri hesaplamanın parçalar üzerinde nasıl yapılacağıyla, diğeri yapay log dosyası üretmekle ilgiliydi. İkisi de "Örnek Sorular" bölümünde ayrıntılı verildi. İkinci sorunun cevabında hoca şunu ekledi: Log dosyası ister gerçek etkinlikten ister yapay olarak oluşturulsun, DFS'ye konduktan sonra geri gidip güncellenmez.

7. Dağıtık Dosya Sisteminin Bileşenleri

7.1 Parçalar ve Parça Sunucuları

DFS'ye bir dosya konduğunda dosya küçük parçalara (chunk) bölünür. Hocanın slayttaki örneği:

- **C dosyası:** 6 parçaya bölünmüş: $C_0, C_1, \ldots, C_5$. Her parça yedeklilik için 2 kopya olarak DFS'ye konmuş. Örneğin C_0 iki sunucuda, C_1 başka iki sunucuda. Toplam $6 \times 2 = 12$ parça kopyası tutulur (ara adım eklendi).
- **D dosyası:** 2 parçaya bölünmüş: D_0, D_1. Her parçanın yine 2 kopyası var. Toplam $2 \times 2 = 4$ parça kopyası (ara adım eklendi).

Veri, parça sunucularında (chunk server) saklanır. Bunlar daha önce anlatılan sıradan sunucudardır, raf kabinlerinde dururlar ve her biri bir Linux makinesi olarak düşünülebilir. Dosya bütün olarak tutulmaz, boyutuna göre parçalara bölünür ve parçalar farklı sunuculara konur.

DFS'ye konan her dosyaya tekil, küresel (global) bir kimlik verilir. Bir kullanıcı C dosyasını istediğinde DFS, C'nin parçalarını farklı parça sunucularında bulur, bunları birleştirip dosyayı yeniden kurar ve kullanıcıya verir. Dosyanın istendiğinde yeniden kurulması da DFS'nin işidir.

Bu dosya üzerinde bir işlem yapmak istediğimizde, örneğin C dosyasında kaç satır olduğunu saymak istediğimizde, hesaplamayı veriye getiririz. DFS, C dosyasının parçalarının hangi

parça sunucularında olduğunu söyler. Sayma kodu bu sunuculara taşınır ve her parça üzerinde çalışır. Her parça sunucusunda bir sonuç oluşur. DFS bu sonuçları alıp tek bir sonuçta birleştirmemize izin verir.

> **Ders sonrası not:** Hoca bu örnekte "file C için altı kopyam var" dedi. Slayttaki örnekte C'nin 6 kopyası değil 6 parçası vardır. Her parçanın 2 kopyası bulunur.

7.2 Parça Boyutu ve Kopya Sayısı

Parça boyutu yapılandırılabilir (configurable). HDFS gibi açık kaynaklı bir DFS kullanıldığında parçalar genelde 16-64 MB büyüklüğündedir. Hoca dersin ilerisinde, bir dosya DFS'ye konduğunda her parçanın genelde 64 MB olduğunu söyledi.

Tutulacak kopya sayısı da yapılandırılabilir. 2 ya da 3 kopya tutmak istediğinizi söyleyebilirsiniz. Slayttaki örnekte bu sayı 2'dir.

7.3 Kopyaların Farklı Kabinlere Konması

HDFS gibi bir DFS'ye bir dosya konduğunda dosya parçalara bölünür ve her parça için 2 ya da 3 kopya tutulur. Ama bu kopyalar farklı raf kabinlerine konur. Nedeni şudur: Kabinlerden birinin anahtarına bir şey olursa, kullanıcı o verinin farklı kabinlerdeki diğer kopyalarına erişebilmelidir. Yani HDFS yalnız çok kopya tutmakla kalmaz, bu kopyaların farklı kabinlerde saklanmasını da sağlar.

7.4 Ana Düğüm (Master Node / NameNode)

Her DFS'de parça sunucularına ek olarak bir ana düğüm (master node) bulunur. Ana düğüm önemlidir, dosya ad alanını sağlamaktan sorumludur.

- Bir dosya DFS'ye konduğunda ona tekil bir kimlik verilir. Kullanıcı daha sonra bu kimlikle dosyayı isteyebilir.
- DFS, parçaları ve dosya kimliklerini izlemek için ana düğümü kullanır. Ana düğüm, dosya kimliklerini ve o dosyaya ait parça kopyalarının nerede olduğunu tutar. Yani dosyaların ya da dosya parçalarının nerede saklandığına dair üst veriyi (metadata) saklar.
- HDFS'te buna NameNode denir. Başka dağıtık dosya sistemlerinde master node denir.
- Parça sunucuları verinin birden fazla kopyasını tutarak yedeklilik ve hata toleransı (fault tolerance) sağlar. Aynı şekilde ana düğümün de birden fazla kopyası tutulur.

Bu noktada 10 dakikalık ara verildi.

7.5 İstemci Kütüphanesi (Client Library)

Aradan sonra hoca üçüncü bileşeni anlattı. Her DFS'nin, DFS'ye erişim sağlayan bir istemci kütüphanesi vardır. DFS'ye bir dosya yazan ya da DFS'den bir dosya okuyan bir uygulama yazdığınızda, kodunuz bu kütüphaneyle konuşur. Kütüphane genelde uygulamanızın bulunduğu makinededir ve doğrudan parça sunucularıyla ya da ana düğümle konuşur.

Hoca bu kısmı şöyle bağladı: DFS, MapReduce tabanlı kodumuzu yazdığımız ve dağıtık dosya işlemini dağıtık depolama sisteminin üzerinde çalıştırdığımız katmandır.

8. Kelime Sayma Problemi

8.1 Problem ve Kullanım Alanları

MapReduce'un ayrıntısına girmeden önce hoca bir uygulama tanımladı. Elimizde çok büyük bir metin belgesi var ve belgede tek tek kelimeler bulunuyor. Her farklı (distinct) kelimenin belgede kaç kez geçtiğini saymak istiyoruz. Uygulama metin dosyasını girdi olarak alır. Çıktı olarak her farklı kelime için o kelimenin kaç kez geçtiğini söyler.

Bu tür sayma uygulamalarının birçok kullanım alanı vardır:

- **Web sunucusu logları:** Popüler URL'leri, yani en çok ziyaret edilen web adreslerini bulmak. Sayma uygulaması her farklı URL'nin kaç kez ziyaret edildiğini söyler.

- **Arama motoru logları:** Arama sorguları loglanabilir. Örneğin 6 ay geriye gidip her sorgu ifadesinin arama motorunda kaç kez kullanıldığını bulmak.

Bu işler genelde tek makinenin belleğine sığmayan büyük veriler üzerinde yapılır ve veriler dağıtık depolarda tutulur. İstenen sonuç, her farklı kelime için bir çifttir: Çiftin ilk elemanı kelime, ikinci elemanı kelimenin metinde kaç kez geçtiğidir.

8.2 Hash Tablosu Neden Yetmez?

Hoca kendi kendine bir soru sordu: Fiziksel bellekte bir hash tablosu tutup belgedeki bütün kelimeleri izleyemez miyim? Hoca bunun iyi bir soru olduğunu söyledi. Ama bizim durumumuzda dosya çok büyüktür ve farklı kelime sayısı da tek bir fiziksel belleğe sığmayacak kadar büyür. Kelime sayılarını tutmak için bir hash tablosu kurmak istesek bile tablo belleğe sığmayabilir.

Slaytta iki durum ayrılıyordu:

- Dosya belleğe sığmayacak kadar büyük, ama kelime-sayı çiftleri belleğe sığıyor.
- Hocanın asıl kastettiği durum: Yalnız dosya değil, kelime-sayı çiftleri de belleğe sığmıyor.

8.3 Linux Komut Zinciri Benzetmesi

İkinci durumda ne yapılabileceğini göstermek için hoca bir Linux komut zinciri gösterdi:

```
words(doc.txt) | sort | uniq -c
```

- **`words`:** Metin dosyasını parametre olarak alır. Metni paragraf olarak tutmak yerine her kelimeyi çıkarıp satır satır yazar.
- **`sort`:** Kelimeleri sözlük sırasına (lexicographical order), yani A harfinden Z harfine doğru sıralar ve sonucu yazar.
- **`uniq -c`:** Sıralı listeyi alır ve her kelimenin listede kaç kez geçtiğini yazar. `-c` argümanı sayıyı verir.

Her komut çıktısını yazar ve bunu bir sonraki komuta girdi olarak verir. Hoca bu üç komutla yapılanı şöyle özetledi: Önce girdi dosyasından bizim için önemli olanı çıkardık. Burada amacımız tek tek kelimeleri saymaktı, bu yüzden kelimeleri çıkarıp satır satır yazdık. Sonra bunları sıraladık. En son da sonuçları istediğimiz şeye, yani her kelimenin sayısına birleştirdik.

> **Ders sonrası not:** `words` gerçek bir Linux komutu değildir. Kitaptaki örnekte dosyadaki kelimeleri satır satır çıkaran varsayımsal bir betiktir. `sort` ve `uniq -c` gerçek komutlardır.

8.4 MapReduce'un Özü

Hoca bu zincirin MapReduce programlama modelinin özünü yakaladığını söyledi. Üç komut MapReduce'un üç adımına karşılık gelir:

- **Map (`words`):** Girdi dosyasını tara ve bizim için önemli olanı çıkar.
- **Group by key (`sort`):** Ara sonuçları grupla, sırala ve karıştır (shuffle). Bu adımı genelde MapReduce programlama kütüphanesi yapar.
- **Reduce (`uniq -c`):** Farklı Map işlemlerinden gelen sonuçları özetle, birleştir ve sonucu yaz.

9. MapReduce Adımları: Map, Group by Key, Reduce

9.1 Girdi: Parça Anahtarı ve Parça İçeriği

Büyük bir metin dosyası DFS'ye konduğunda DFS onu küçük parçalara böler (genelde 64 MB) ve parçaları birden fazla veri düğümüne koyar. Slayttaki örnekte C dosyası C_0'dan C_5'e kadar 6 parçaya bölünmüştür.

DFS dosya kimliği ile parça kimliği arasındaki ilişkiyi izlemek zorundadır. Bu yüzden her parçaya da bir anahtar (key) verir. Böylece her parça bir anahtar-değer (key-value) çifti olarak düşünülebilir: anahtar parçanın kimliği, değer parçanın içeriğidir. MapReduce programı DFS'nin sağladığı bu dağıtık platformun üzerinde çalışır.

9.2 Map Adımı

Yazdığımız ilk fonksiyon Map'tir. Map her parça sunucusunda çalışır. Anahtar-değer çiftinin değer kısmını, yani parçanın kendisini işler ve bizim için önemli olanı çıkarır. Kelime saymada önemli olan kelimelerdir.

Map her parça için o parçadaki her kelimeyi çıkarır. Çıkardığı her kelime için anahtar olarak kelimeyi, değer olarak 1 yazar. Slayttaki örnekte ilk anahtar "apple", ikincisi "orange" gibidir. Map fonksiyonu her parça için aynıdır.

Hoca bunun ne kadar basit olduğunu vurguladı: Map parçaya gider, bütün kelimeleri çıkarır, her birini anahtar olarak yazar ve yanına değer olarak 1 yazar. Map girdiyi kayıt kayıt tarar ve bizim önemsediğimiz şeyi çıkarır. Okunan her kelimenin sayısı 1'dir. Map bitince ara anahtar-değer çiftleri (intermediate key-value pairs) oluşur.

9.3 Group by Key Adımı

Sonraki adım "anahtara göre gruplama"dır (group by key). Bu adımı MapReduce kütüphanesinin kendisi yapar. Örneğin Hadoop MapReduce kütüphanesi kullanılıyorsa bu adımı Hadoop yapar. Kütüphane aynı anahtara sahip çiftleri bir araya getirir.

Hocanın örneğinde "orange" anahtarı için bütün "orange" çiftleri bulunmuş ve bir araya getirilmiştir. Üç tane bulunmuş, bu yüzden aynı anahtar ve üç değer var. Başka bir kelime için iki tane bulunmuş ve onlar da bir araya getirilmiş.

> **Hocanın vurgusu:** Group by key adımı hiçbir işlem yapmaz. Yalnız aynı anahtarlı çiftleri bir araya getirir.

9.4 Reduce Adımı

Son adım Reduce'tur. Hoca bir programcı olarak bizim iki fonksiyon yazdığımızı vurguladı: Map'i de Reduce'u da biz yazarız. DFS'de MapReduce işlemi yapmak istediğinizde yazmanız gereken iki fonksiyon vardır: `map` ve `reduce`.

Problemimiz farklı kelimeleri saymaktı. Reduce fonksiyonu aynı anahtara karşılık gelen değerleri sayar. "orange" örneğinde 1, 2, 3 değer vardır, bu yüzden Reduce 3 yazar. Diğer Reduce'ta 1 ve 1 vardır, toplam 2'dir.

Hoca bütün akışı şöyle bağladı: Her parça sunucusunda bir Map çalışır. Map'ler bitince ara anahtar-değer çiftlerini aynı parça sunucusuna yazar. Sonra bu çiftler aynı anahtarlara göre gruplanır. Gruplama bitince Reduce işleri aynı anahtara karşılık gelen değerleri alır ve bir sonuç üretir.

> **Ders sonrası not:** Hoca 01:26:02 civarında gruplamayı "dağıtık dosya sistemi yapar" dedi. Dersin geri kalanında ve slaytta bu iş MapReduce kütüphanesine (Hadoop, Spark vb.) aittir. DFS yalnız girdiyi parçalara bölüp saklar.

9.5 Map ve Reduce'un Biçimsel Tanımı

Hoca slayttaki biçimsel tanımı açıkladı:

- **Girdi:** Bir anahtar-değer çiftleri kümesi.
- **Map:** $\text{Map}(k, v) \rightarrow \langle k', v' \rangle^*$. Bir anahtar-değer çifti alır. Buradaki değer, dosyaya ait küçük bir parçadır. Çıktı olarak ara anahtar-değer çiftleri üretir. Yıldız işareti, Map'in birden çok ara çift üretebileceğini gösterir.
- **Reduce:** $\text{Reduce}(k', \langle v' \rangle^*) \rightarrow \langle k', v'' \rangle^*$. Bir ara anahtarı ve ona karşılık gelen değerlerin listesini alır. Yıldız bu kez değerlerin birden çok olduğunu gösterir. Her ara anahtar için bir sonuç üretir. Örneğimizde bu sonuç sayıdır.

Değerler üzerinde ne işlem yapılacaksa Reduce onu uygular ve bir sonuç üretir. Birden çok ara anahtar olduğu için birden çok Reduce sonucu olur.

9.6 Örnek Akış: Büyük Metin Dosyası

Hoca bir sonraki slaytta bütün akışı tek resimde gösterdi. Büyük metin dosyası parçalara bölünmüştü. Map, önemsediğimiz şeyi, yani kelimeleri tek tek çıkarmış ve her birine değer olarak 1 yazmıştı. Bunu diğer parçalar için de yapmıştı.

Group by key aynı kelimeleri bir araya getiriyordu. Örneğin "crew" kelimesinden biri bir parçada, bir diğeri başka bir parçadaydı ve bir araya getirilmişti. "space" kelimesinden yalnız bir tane vardı, bu yüzden tek bir çift görünüyordu. "the" için de aynı anahtara sahip bütün çiftler bir araya gelmişti.

Gruplama bitince kütüphane bu anahtar-değer çiftlerini Reduce programının bulunduğu yere taşır. Hocanın örneğinde 4 parça sunucusu vardı:

- Her parça sunucusu için bir Map olabilir.
- Reduce sayısı 4 olabilir, 3 de olabilir. Reduce sayısı Map sayısından az olabilir.
- Map sayısının Reduce sayısına eşit olması gerekmez. Ama yaygın yaklaşım, Map sayısının dosya parçalarının bulunduğu parça sunucusu sayısına eşit olmasıdır.

Slayttaki örnekte 3 Reduce vardı. Kütüphane gruplanmış çiftleri Reduce'lardan birine taşıyordu: bir grup birinci Reduce'un yerine, bir grup ikincinin yerine, bir grup üçüncünün yerine. Sonra her Reduce aynı kelimeye ait değerleri sayıyordu. İki sonuç birinci Reduce'un, biri ikincinin, biri de üçüncünün sonucuydu.

Hocanın 9.3 ve 9.4'te anlattığı "orange" örneği, üç adımda şöyle özetlenir:

```
Map çıktısı:   (orange, 1) (orange, 1) (orange, 1)  ... (diğer kelime, 1) (diğer kelime, 1)
Group by key:    (orange, [1, 1, 1])                        (diğer kelime, [1, 1])
Reduce:          (orange, 3)                                 (diğer kelime, 2)
```

9.7 Sözde Kod

Hoca Hadoop gibi bir kütüphanede kodun nasıl göründüğünü gösterdi. Programcı Map ve Reduce fonksiyonlarını yazmaktan sorumludur:

```
map(key, value):
    // key: belge adı (parçanın anahtarı)
    // value: belge metni (parçanın kendisi)
    for each word w in value:
        emit(w, 1)

reduce(key, values):
    // key: kelime
    // values: sayılardan oluşan bir yineleyici
    result = 0
    for each count v in values:
        result += v
    emit(key, result)
```

Map fonksiyonu bir anahtar-değer çifti alır. Anahtar veri parçasının anahtarı, değer parçanın kendisidir. Başta DFS'ye büyük metin dosyası verilmiş ve DFS onu küçük metin dosyalarına bölmüştü. Değer bunlardan biridir. Kod, bu küçük metin parçasındaki her kelime için ilk elemanı kelime, ikinci elemanı 1 olan bir çift yazar. Map ara anahtar-değer çiftlerini oluşturur.

Map işini bitirince bütün ara çiftler gruplanır ve Reduce'un çalışabileceği yere taşınır. Reduce her anahtar için değerler üzerinde bir "for each" döngüsü döner, değerleri toplar ve (anahtar, sonuç) çiftini yazar. Reduce bunu kendisine verilen her ara anahtar için yapar.

10. Başka MapReduce Örnekleri

10.1 Alan Adına Göre Toplam Boyut

Hocanın ikinci örneği: Elimizde her satırında bir URL, o URL'nin boyutu ve tarihi olan büyük bir derlem (corpus) var. Her alan adı (host) için toplam bayt sayısını bulmak istiyoruz. Hoca

örnek olarak kendi bölümümüzün web sitesini verdi: bölüme ait bütün URL'leri alıp boyutlarını toplamak.

- **Map:** Her URL için önce host adını bulur. Ara anahtar-değer çifti (host adı, sayfa boyutu) olur. Böylece birden çok ara çift oluşur. Bazılarının anahtarı Yıldız'ın bilgisayar mühendisliği bölümüne, bazılarınınki Boğaziçi'nin bilgisayar mühendisliği bölümüne aittir. Her web sayfasının boyutu farklı olduğu için değerler farklıdır.
- **Group by key:** Kütüphane aynı host adlarını gruplar ve Reduce'un bulunduğu yere taşır.
- **Reduce:** Sayma yerine toplama yapar. Aynı host adına ait bütün boyutları toplar ve bir sonuç üretir.

10.2 Beş Kelimelik Dizileri Sayma

Üçüncü örnek: Büyük bir belge derleminde her beş kelimelik dizinin kaç kez geçtiğini saymak. Tek tek kelimeler yerine beş kelimelik diziler sayılır.

- **Map:** Belgeden beş kelimelik dizileri çıkarır ve her ara çift için değer olarak 1 yazar. Hocanın örneği: İlk dizi "the crew of the space" olur, değer 1. Sonra bir sonraki diziye geçilir: "crew of the space shuttle", değer 1. Böylece bütün beş kelimelik diziler çıkarılır.
- **Group by key:** Kütüphane aynı beş kelimelik dizileri gruplar ve Reduce'un bulunduğu yere taşır.
- **Reduce:** Kelime saymadaki gibi sayma yapar.

"the crew of the space shuttle" gibi altı kelimelik bir metinden $6 - 5 + 1 = 2$ beş kelimelik dizi çıkar (ara adım eklendi). Bunlar hocanın söylediği iki dizidir.

Hoca bunların MapReduce'u anlamak için güzel ve basit örnekler olduğunu, Hadoop ya da Spark üzerinde çalışmaya başlayınca bu basit uygulamaların kendi başımıza yazılabileceğini söyledi.

11. Arka Planda Olanlar

11.1 Map'lerin Parça Sunucularına Gönderilmesi

Hoca sonra "kaputun altına" baktı: DFS dağıtık MapReduce'a nasıl yardım ediyor? Büyük belge DFS'ye verilir. DFS onu parçalara böler ve farklı sunuculara koyar. Her parça sunucusunda bir Map fonksiyonu vardır. Map'i kodunuzda siz yazarsınız. Hadoop ya da Spark gibi büyük veri kütüphaneleri kodunuzu alır, parça sunucularının bulunduğu yere gönderir ve orada çalıştırır.

Kelime sayma probleminde Map yalnız kelimeleri çıkarır, başka hiçbir şey yapmaz. Slayttaki örnekte bir Map "key 1, key 1, key 2" çıkarmıştı. Bir parçada veri yoktu, bu yüzden o Map hiçbir şey üretmemişti. Slaytta toplam 7 Map vardı ve her biri bir veri parçası üzerinde çalışıyordu.

11.2 Ara Çiftlerin Yerel Diske Yazılması ve Taşınması

Ara anahtar-değer çiftleri parça sunucularının bulunduğu yerde üretilir ve bu sunucuların yerel dosya depolarına yazılır. Ardından kullanılan MapReduce kütüphanesi bu ara çiftleri alır ve Reduce fonksiyonlarının bulunduğu yere taşır. Slayttaki örnekte 5 Reduce fonksiyonu vardı ve farklı parça sunucularında olabilirlerdi. Group by key adımı çiftleri ilgili Reduce'un bulunduğu yere taşıyordu. Örneğin "key 1" çiftlerinden ikisi bir yerden, biri başka bir yerden, biri de başka bir yerden aynı Reduce'a taşınmıştı.

Burada bir öğrenci Reduce işini neden kütüphanenin yapmadığını sordu. Cevap "Örnek Sorular" bölümündedir.

11.3 Bölümleme Fonksiyonu (Partitioning Function)

Bir sonraki slaytta 5 parça sunucusu vardı. Her sunucuda farklı sayıda veri parçası bulunuyordu: birinde 3, diğerlerinde 2. Her veri parçası için bir Map vardı. Bu, bir parça sunucusunda birden çok Map olabileceği anlamına gelir, ama her veri parçası için bir Map olur. Map bitince ara çiftleri yazar ve bunlar yerel dosya sisteminde saklanır.

Sonra bölümleme fonksiyonu devreye girer. Bu fonksiyon Hadoop, Spark ya da Apache Flink gibi MapReduce kütüphanelerine aittir. Örnekte Reduce işlerinin çalışacağı 2 sunucu seçilmişti. Birinde 3, diğerinde 2 Reduce işi vardı ve her Reduce'un bir numarası vardı (1'den 5'e).

> **Hocanın vurgusu:** Bölümleme fonksiyonu ara çiftin anahtarını bir hash fonksiyonuna sokar. Hash'in çıktısı mevcut Reduce'lardan biridir ve çiftin hangi Reduce'a gideceğini belirler. Kütüphane çifti o Reduce'un bulunduğu makineye taşır.

Örnekte anahtarı "key 1" olan bütün çiftler Reduce 1'e gönderilmişti. 4 tane "key 1" çifti vardı. Reduce bunları girdi olarak alır ve sonuç olarak (key 1, 4) çiftini üretir. Reduce 3 de aynı şeyi yapar: anahtarı K3 olan bütün ara çiftler oraya taşınmıştır. Diğer Reduce'lar için de durum aynıdır.

Bu örnek kurulumda 5 parça sunucusunun 3'ü Map, 2'si Reduce için kullanılmıştı. Ama hoca bunun zorunlu olmadığını söyledi. Reduce'lar Map'lerin çalıştığı sunucularda da çalışabilir. Örneğin 3 Reduce, Map'lerin çalıştığı yerde çalıştırılabilirdi. Reduce görevlerinin ayrı parça sunucularında olma zorunluluğu yoktur.

11.4 MapReduce Ortamının Görevleri

Hoca slayttaki listeyi madde madde açıkladı. Programcının işi Map ve Reduce'u yazmaktır. Gerisini ortam üstlenir:

- **Girdi verisini bölümlemek:** Başta verilen girdi dosyasının parçalara bölünüp DFS'deki parça sunucularına konması işini, MapReduce'un üzerinde çalıştığı DFS halleder.
- **Programın makineler üzerinde çalışmasını zamanlamak:** Map ve Reduce fonksiyonları yazılıp çalıştırma başlatıldığında, kütüphane önce Map'leri çalıştırıp ara çiftleri oluşturur. Ardından ara çiftleri ilgili Reduce'un bulunduğu yere taşır. Bütün veri gelince Reduce'ları çalıştırır. En son Reduce sonuçlarını DFS'ye yazar. Bunları sırasıyla yaptıran yine MapReduce kütüphanesidir.
- **Group by key adımını yürütmek:** Kendisine taşınan çiftlerin gruplanmasını kütüphane yapar.
- **Düğüm arızalarını yönetmek:** Bir düğüm bozulursa, o düğümdeki veri parçalarının kopyalarının nerede olduğu NameNode üzerinden öğrenilir ve Map ya da Reduce'lar orada çalışmaya devam eder. Çalıştırma başladıktan sonra arka planda arızalar olsa bile, uygulama tamamlanana kadar koda yeniden müdahale etmek gerekmez.
- **Makineler arası iletişimi yönetmek:** Map'in ürettiği ara çiftler saklanır. Bölümleme fonksiyonu anahtarın hangi Reduce'a gideceğini belirler. Çiftlerin o Reduce'un bulunduğu sunucuya taşınması da kütüphanenin işidir.

Programcı olarak biz MapReduce'u gerçekleştiren bu kütüphanelere yalnız Map ve Reduce içinde hangi kodun çalışacağını yazarız. Hoca bunu dersin başına bağladı: Dağıtık hesaplamanın karmaşık olduğunu söylemiştik ve ayrıntısını şimdi gördük. Bu karmaşıklıkla biz değil, kütüphane uğraşır. Biz yalnız yazmak istediğimiz kodun mantığıyla uğraşırız. Programlama modeli bunu sağlar.

11.5 Veri Akışı: Girdi, Ara Sonuç, Çıktı

- **Girdi ve son çıktı DFS'dedir:** MapReduce'un girdisi DFS'de saklanır, yani her veri parçasının birden çok kopyası vardır. Zamanlayıcı (scheduler) Map görevlerini girdi verisinin fiziksel olarak saklandığı yere yakın zamanlamaya çalışır. MapReduce'un çıktısı da DFS'de saklanır. Böylece Reduce'a bir şey olsa bile sonucu birden çok kopya olarak saklanmış olur.
- **Ara sonuçlar yerel diskte tutulur:** Ara sonuçlar Map ve Reduce işçilerinin (worker) yerel depolarında saklanır.

  > **Hocanın vurgusu:** Ara anahtar-değer çiftleri DFS'ye yazılsaydı, HDFS her parçayı yaptığı gibi bunları da 2 ya da 3 kopya halinde saklardı. Oysa ara çiftler yalnız MapReduce'un iç işleyişi için önemlidir, iş bitince bir anlamları kalmaz. DFS'ye yazmak hem fazla yer kaplar hem de MapReduce'un toplam süresini artırır. Bu yüzden ara çiftler yalnız yerel diske yazılır.

Bir MapReduce görevinin çıktısı çoğu zaman başka bir MapReduce görevinin girdisi olur.

11.6 Kendi HDFS Kümenizi Kurmak

Hoca yerel disk ile HDFS alanı farkını somut bir örnekle anlattı. Bir masaüstü bilgisayarınız ve bir sunucunuz olsun. Bu ikisi üzerinde iki düğümlü bir DFS kurmak istiyorsunuz. Hadoop'u indirip HDFS'i kurdunuz ve biri masaüstü, biri sunucu olan iki düğümlü bir HDFS kümesi oluşturdunuz.

Hem sunucuda hem masaüstünde yine kendi yerel diskiniz olur. Yerel diskin yanında bir de HDFS'in kapladığı dağıtık depolama alanı olur. HDFS size bu iş için ne kadar alan kullanmasını istediğinizi sorar. Siz örneğin "30 GB veriyorum" dersiniz ve hem sunucuda hem masaüstünde 30 GB'lık alanı yapılandırırsınız. HDFS bir veri sakladığında bu alanları kullanır. Geri kalan alan ise makinenin yerel diskidir. Yani bir veriyi saklamak istediğinizde iki seçeneğiniz vardır: HDFS'e ya da yerel diske yüklemek.

MapReduce'a verdiğimiz girdiyi HDFS'e koyarız, çünkü HDFS onu küçük parçalara ayırıp birden çok kopya olarak tutar. Kodlar çalıştıktan sonra oluşan sonuçları da HDFS'e koyarız. Ama MapReduce çalışırken oluşan ara çiftleri HDFS'e koymayız, yerel diskte tutarız. İş bitince yerel diskten zaten silinirler.

12. Ana Düğümün Koordinasyonu ve Arızalar

12.1 Ana Düğümün Görevleri

Parça sunucularına ek olarak bir ana düğüm vardır ve koordinasyonu o üstlenir:

- Parça sunucularını, Map ve Reduce görevlerini izler. Her görevin durumu üç değerden biridir: boşta (idle), sürüyor (in progress), tamamlandı (completed).
- Boşta olan görevler, işçiler uygun hale geldikçe zamanlanır.
- Bir Map görevi bitince ana düğüme, her Reducer için ürettiği ara dosyaların konumunu ve boyutunu gönderir.
- Ana düğüm bu bilgiyi Reducer'lara iletir (push).

Hoca bu akışı Türkçe olarak adım adım anlattı:

1. Her Map işini bitirince, yani ara çiftleri üretip yerel diskine yazınca, ana düğüme bir mesaj gönderir. Mesajda şunu söyler: Şu numaralı Map görevi için, kendi bulunduğu makinede, şu isimli dosyada anahtar-değer çiftlerini bitirdim.
2. Map, bölümleme fonksiyonunu kullanarak çiftlerin hangi Reduce'a gideceğini de belirlemiştir. "Bitti" sinyalini gönderirken hangi dosyanın hangi Reduce'a gideceğini de söyler.
3. Ana düğüm ilgili Reducer'a bir mesaj gönderir: Şu makinede sana gelmesi gereken şu isimli bir dosya var.
4. Reducer gidip veriyi alır, kendi makinesine kopyalar, Reduce işlemini yapar ve sonucu DFS'ye yazar.

Ana düğüm bu sırada bilgisi olan bütün Map ve Reduce işçilerine düzenli aralıklarla ping atar ve hayatta olup olmadıklarına bakar. Bir Map ya da Reduce bozulmuşsa, aynı işi verinin kopyasının bulunduğu başka bir sunucuda yeniden çalıştırır. Her Map'in işlediği veri parçasının birden çok kopyası olduğu hatırlanırsa, bu mümkündür.

> **Ders sonrası not:** Hoca MapReduce koordinatörünü "master", "name node" ve "name server" diye adlandırdı. Bu derste hepsi aynı koordinatör düğümü kastediyor. Hadoop'ta büyük olasılıkla HDFS'in üst verisini NameNode tutar, MapReduce görevlerini ise ayrı bir koordinatör (eski sürümlerde JobTracker, yenilerinde YARN ResourceManager) yönetir.

12.2 Map İşçisi Arızası

Map'in çalıştığı makinede bir hata olursa, Map görevi ister tamamlanmış ister sürüyor olsun, ana düğüm görevin etiketini boşta (idle) yapar. Sonra bu görev, üzerinde çalıştığı veri parçasının bulunduğu başka bir düğümde yeniden çalıştırılır.

Tamamlanmış Map'in bile yeniden çalıştırılmasının nedeni derste ayrıca söylenmedi. Ama hocanın az önce anlattığından çıkar: Map'in çıktısı DFS'ye değil, bozulan makinenin yerel diskine yazılmıştır. Makine bozulunca bu çıktıya erişilemez (ara adım eklendi).

12.3 Reduce İşçisi Arızası

Bozulan bir Reduce işçisiyse ve görevi sürüyorsa, görev boşta yapılır ve başka bir düğümde yeniden çalıştırılır. Ama görev tamamlanmışsa yeniden çalıştırmak gerekmez. Çünkü tamamlanmış bir Reduce çıktısını mutlaka HDFS'e yazmıştır. HDFS'e yazılan bir şeyin birden çok yerde kopyası vardır. Yani sonuç yalnız bozulan sunucuda değil, başka bir sunucuda da bulunur.

> **Hocanın vurgusu:** Tamamlanmış Reduce yeniden çalıştırılmaz, çünkü çıktısı DFS'de kopyalanmıştır. Map ise tamamlanmış olsa da yeniden çalıştırılır.

Burada bir öğrenci "fonksiyonun bozulması"nın ne demek olduğunu sordu. Cevap "Örnek Sorular" bölümündedir.

12.4 Ana Düğüm Arızası

Ana düğüm bozulursa ne olur? HDFS gibi dağıtık dosya sistemlerinde ana düğümün de kopyası tutulduğu için işlem diğer ana düğüm üzerinden devam eder. Ama yalnız tek bir ana düğüm varsa, çalıştırılan MapReduce görevi iptal edilir (abort) ve görevi başlatan istemciye görevin başarıyla tamamlanamadığı bildirilir.

13. Map ve Reduce Görevi Sayısı

Hoca son olarak şunu sordu: 5 düğümlü, MapReduce çalıştırabileceğim bir HDFS ortamı kurdum ve kodlarımı üzerinde çalıştıracağım. Map ve Reduce sayısı yapılandırılabilir mi? Evet, yapılandırılabilir.

- **Map sayısı (M):** Genelde kümedeki düğüm sayısından fazla tutulur. Hocanın örneği: Word count yapılan dosyanın 7 parçası varsa, her veri parçası için bir Map, yani 7 Map çalıştırmayı tercih ederim. Bu sayı genelde toplam düğüm sayısının üzerindedir.
- **Reduce sayısı:** Genelde Map sayısından çok daha azdır. Az önceki örnekte 7 Map'e karşılık 5 Reduce vardı. Bu sayı 4 ya da 3 de olabilirdi.

Bir HDFS kurduktan sonra kaç parça sunucusu kullanılacağı, birden fazla ana düğüm tutulup tutulmayacağı, bir MapReduce işinde kaç Map ve kaç Reduce olacağı yapılandırılıp kodlar öyle çalıştırılabilir.

Örnek Sorular

Bu derste ekrandan alınmış soru yoktur. Aşağıdaki sorular derste sözlü sorulan sorulardır.

Soru 1 (derste sözlü, 00:30:27)

Soru: Hesaplamayı, veriyi taşımak yerine verinin tutulduğu yerde yapalım dedik. Veri parçalar halindeyken hesaplamayı nasıl yapacağız? Her durumda bir şeyleri taşımak zorunda kalıyor muyuz?

Cevap: Hoca bir log dosyası örneği verdi. Dosyanın her satırı bir log etkinliğidir ve dosyada milyonlarca satır vardır.

- Dosya 4 parçaya bölünür. Her parçada yine log kayıtları vardır.
- Parçalar farklı sunuculara konur ve yedeklilik için her parçanın 3 kopyası tutulur. Toplam $4 \times 3 = 12$ parça kopyası olur (ara adım eklendi).
- Dosyadaki log kayıtlarını sayan bir programımız var. Kodun boyutu veriye göre çok küçüktür. Sayma kodu parçaların bulunduğu 4 sunucuya taşınır ve orada çalışır.
- Her kod kendi parçasındaki log kayıtlarını sayar. 4 sunucuda 4 ara sonuç oluşur.
- MapReduce modeli ya da onu kullanan bir kütüphane bu 4 sonucu alır ve tek bir sunucuda birleştirir. Böylece dosyadaki bütün etkinliklerin sayısı bulunur.

Yani taşınan veri değil, her sunucudaki hesaplamanın sonucudur. Hoca konunun ilerleyen kısımlarda daha iyi anlaşılacağını ekledi.

Soru 2 (derste sözlü, 00:37:03)

Soru: Yapay log dosyaları oluşturabilir miyiz? Bunun için bir program var mı?

Cevap: Evet, yapay log dosyası oluşturulabilir. Ne kadar uygun olduğu, dosyayı hangi amaçla kullanmak istediğinize bağlıdır. Log etkinlikleri genelde kullanıcıların çevrim içi ne yaptığını bulmak için kaydedilir. Hoca asıl noktayı şöyle bağladı: Log dosyası nasıl oluşturulursa oluşturulsun, ister gerçek etkinlikten ister yapay olarak, DFS'ye konduktan sonra geri gidip güncellenmez. Büyük veri nadiren güncellenen veridir.

Soru 3 (derste sözlü, 01:29:02)

Soru: Aynı kelimeleri Map fonksiyonunda sayamaz mıyız?

Cevap: Evet, yapılabilir. Hoca bunun birleştirici (combiner) fonksiyonu olduğunu ve ayrıca konuşulacağını söyledi.

- Ara çiftlerin Reduce'un çalışacağı yere taşınması için ağ üzerinden bir kopyalama yapılır. Bunu programcı değil, Hadoop ya da Spark gibi kullanılan kütüphane yapar.
- Taşınacak değer sayısı çok fazla olursa bu taşıma zaman alır.
- Bunu önlemek için kelime sayımının bir kısmı Map'in bulunduğu yerde yapılabilir. Ama Map tarafında yalnız o parçada yerel olarak karşılaşılan kelimeler sayılabilir.
- Hocanın sayısal örneği: C_0 parçasından 10 kelime çıktı ve bunların 4'ü farklı. Birleştirici, Map'in bulunduğu yerde bu 10 çifti 4 çifte indirir ve sayma işinin bir kısmını orada yapar. Sonra Reduce'un bulunduğu yere 10 değil 4 çift taşınır.
- Kazanç, değerleri taşırken ağda harcanan süredir.

Soru 4 (derste sözlü, 01:50:46)

Soru: Reduce işlemini neden kütüphane yapmıyor?

Cevap: MapReduce yazarken yazmamız gereken iki fonksiyon vardır: Map ve Reduce. Reduce'ta ne yapılacağı probleme bağlıdır, bu yüzden içini programcı yazar.

- **Host boyutu örneği:** Map her sayfanın hangi alan adına ait olduğunu bulup (alan adı, sayfa boyutu) çifti yazıyordu. Reduce'ta aynı alan adına sahip sayfaların toplam boyutu isteniyordu, yani toplama yaptırılıyordu.
- **Kelime sayma örneği:** Map bir veri parçasındaki kelimeleri hiçbir işlem yapmadan yalnız yazıyordu. Reduce aynı kelimeye karşılık gelen değerleri topluyordu.
- Aynı ara anahtara karşılık gelen veriyi nasıl birleştireceğimizi, yani toplama mı, çarpma mı, bölme mi, hangi işlemi yapacağımızı Reduce'ta biz kodlamak zorundayız.
- Program çalıştırılınca verdiğimiz Reduce kodunu yine MapReduce kütüphanesi çalıştırır. Map'ten başlayıp Reduce'un sonuna, çıktı oluşana kadar bütün işlemi kütüphane yürütür. Ama Map'in ve Reduce'un ne olacağını biz veririz.
- Group by key işlemini kütüphanenin kendisi yapar. Ara çiftleri anahtara göre sıralamak ve sıralanmış çiftleri Map'in bulunduğu sunucudan Reduce'un bulunduğu sunucuya taşımak kütüphanenin işidir. Hoca bu adımın da yapılandırılabildiğini, buraya yönlendirme kodu yazılabileceğini ve bunun ayrıca konuşulacağını söyledi.

Soru 5 (derste sözlü, 02:14:21)

Soru: Fonksiyonun bozulması (fail olması) ne demek? Üzerindeki sunucunun bozulması mı?

Cevap: Aynen öyle, fonksiyonun çalıştığı sunucunun bozulması demektir.

- Örneğin bir sunucuda 3 Map fonksiyonu çalışıyor. Ana düğüm bunlara sürekli ping atar.
- Çalışmadıklarını görürse bunun anlamı o sunucuda bir şey olduğu ve sunucunun bozulduğudur.
- Bu 3 Map tamamlanmış ya da sürüyor olsun, ana düğüm hepsinin durumunu boşta (idle) yapar.
- Bu 3 Map'i, veri parçalarının kopyalarının bulunduğu başka parça sunucularında yeniden çalıştırır.

- Reduce için de benzer kural geçerlidir: Reduce tamamlanmışsa yeniden çalıştırmaya gerek yoktur, çünkü sonucu HDFS'e yazılmıştır ve başka bir sunucuda da vardır. Sürüyorsa başka bir sunucuda yeniden çalıştırılır.

Soru 6 (derste sözlü, hocanın sorusu, 02:16:23)

Soru: 5 düğümlü bir HDFS ortamı kurdum ve MapReduce kodlarımı üzerinde çalıştıracağım. Map ve Reduce sayıları yapılandırılabilir mi?

Cevap: Evet. Map sayısı genelde düğüm sayısından fazla tutulur, her veri parçası için bir Map çalışır. 7 parçalı bir dosyada 7 Map olur. Reduce sayısı genelde Map sayısından azdır. Örnekte 7 Map'e karşılık 5 Reduce vardı, 4 ya da 3 de olabilirdi.

Dersin Toparlaması

Bu derste büyük verinin neden tek makinede işlenemediği görüldü. CPU yalnız bellekteki veriyle çalışır ve 200 TB'lık bir veriyi tek diskten okumak bile 46 günden uzun sürer. Çözüm, sıradan Linux sunuculardan oluşan, raf kabinleri ve anahtarlarla bağlanmış kümelerdir. Ama kümelerde düğüm arızaları her gün olur, ağ bir darboğazdır ve dağıtık programlama karmaşıktır. Dağıtık dosya sistemi (HDFS, GFS) dosyaları 16-64 MB'lık parçalara bölüp her parçayı 2-3 kopya halinde farklı kabinlerde saklar. Parçaların yerini ana düğüm (NameNode) tutar, uygulamalar istemci kütüphanesiyle erişir.

MapReduce bu altyapının üzerinde hesaplamayı veriye taşır. Programcı yalnız `map` ve `reduce` fonksiyonlarını yazar. Map her parçadan önemli bilgiyi ara anahtar-değer çiftleri olarak çıkarır. Kütüphane bu çiftleri bölümleme fonksiyonuyla Reduce'lara dağıtır ve anahtara göre gruplar. Reduce her anahtarın değerlerini birleştirir. Girdi ve çıktı DFS'de, ara sonuçlar yerel diskte tutulur. Bu yüzden bozulan bir makinedeki Map yeniden çalıştırılır, tamamlanmış Reduce ise çalıştırılmaz. Hoca gelecek derste MapReduce iyileştirmeleriyle (refinements), özellikle birleştirici (combiner) fonksiyonuyla devam edeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.